Introduction

Developed by Fred Glover in the 1970's, tabu search is a meta-heuristic designed for combinatorial problems that is based on a restrained exploration of the search space. The algorithm, in its canonical form, explores the whole neighborhood of a single solution in each iteration, making it a single-solution algorithm (rather than population-based). Having defined and explored the whole neighborhood with a specific move operator and encoding, the algorithm proceeds to make the best possible move.

The main characteristic of this algorithm is the tabu list that restricts the moves. To avoid a greedy behavior, tabu search saves movements that would make it to go back on the search path. This way the algorithm does not loop around local optimums in its iterations. In its canonical form, the tabu list have a fixed size and the restrictions of movements saved in this list expires given a certain number of executed iterations. This aspect of the algorithm is often referred as short-term memory. The degree of restrictions to the moves not only depends on the size of the tabu list but also on its entries. Depending on how the movement is saved, the possibilities of the algorithm could be greatly altered.

A common implementation of this algorithm is the aspiration criterion. This characteristic, often referred as long-term memory, saves a certain aspect of the search, and gives the algorithm the possibility to make a restricted move if the criterion is met. The usual selection of the aspiration criteria is the best solution so far. For example, if a restricted solution has a better objective function than the best solution visited so far by the algorithm, the tabu list is ignored.

Tabu search is good algorithm for problem with a clear structure on the neighborhood. Therefore, its performance depends greatly on the problem to solve, the encoding and move operator selected. For this homework, the problem to solve is the quadratic assignment problem (QAP) with 20 departments. This problem consists of locating 20 departments in a grid with specific distances and flows and calculating the overall costs considering the flows that must be satisfied and the distances that depends on the selected locations.

Considering all this, in this report an analysis on the initial solutions is explored. Then, different levels for the tabu size are executed. Finally, several variations of the algorithm are implemented, and conclusions are described for the executions of this homework.

General definitions

To execute the tabu search meta-heuristic several definitions must be made considering the problem and the canonical implementation of the algorithm. The first one is the encoding of the problem. For the 20 departments QAP, the selected encoding consists of lists of the departments in which the index of the list indicates the location of the department on the grid. This encoding was chosen because it is a simple way to represent a solution in a symmetric location problem like the QAP. Also, it allows easy implementation of moves operator that doesn't have to keep track of the feasibility of the solutions and satisfy with the desirable characteristics of and encoding (one to one representations and can reach every solution eventually).

Considering this encoding, the move operator selected was a two elements swap on the list of strings. As mentioned, this swap allows keeping solutions feasible for the problem while exploring the search space in a conservative way. The neighborhood defined by this move operator are all the solutions that can be reached by a single two-element-swap from the current solution of the algorithm. In figure 1, an example of an encoded solution for a 3 department QAP and its respective neighborhood are shown.

$$[2,0,1] => [1,0,2] - [0,2,1] - [2,1,0]$$

Figure 1: Example of (3) department QAP encoding and its neighborhood with the selected move operator

Another important aspect to define for the implementation of the algorithm is the tabu entry. This is what aspects of the solutions, or the movements are saved in the tabu list for future restrictions. For the implementation of this homework, the tabu entry was defined by the pairs of solutions involved in the 2-element swap. This way the algorithm would not return through the search path if the next neighborhood doesn't find a better solution. Another aspect of the tabu list is how it updates itself. In this case, the tabu list is based on recency that translates in eliminating the oldest entries when the tabu size limit is reached.

Also, considering that the only random operator (in the canonical implementation) of the algorithm is the initial solution from where the algorithm starts the search, the initial solution was built with a random sampling of the nodes given a specific random seed.

Finally, the termination criteria for the executions of the runs were determined in 500 hundred iterations of the algorithm for all the executions of this homework. Considering the computational costs, this value was selected to ensure longer runs of the algorithm and explore each of the variations implemented (especially the ones like the diversification and partial neighborhoods implementations that include random operators).

All these definitions are summarized in the following table.

Table 1: General definitions

Definition	Selection	Justification
Encoding	List of strings with index as chosen position	Satisfy conditions for a good encoding
Move operator	Two-element-swap	Ensures feasibility of the
Tabu entry	Pair of solutions involved in the move	Restrictive enough to diversify the search
Initial solution	Random based on seed	Test the algorithm's behavior
Termination criteria	500 iterations	Enough iterations to test the different random strategies

Preliminary runs

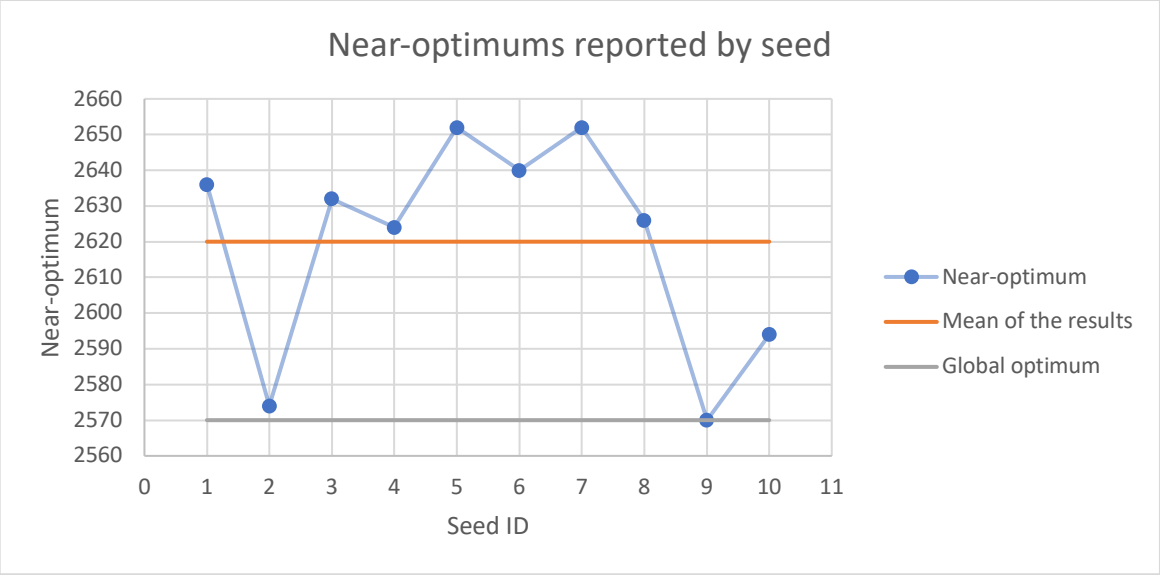
Parameter definition

The preliminary run's objective is to test the algorithm sensitivity to the randomness of the initial solution. This parameter for the execution of the algorithm is the only source of variation in the canonical form of the algorithm. Once defined the initial solution, the search that the canonical tabu search executes is completely deterministic. Therefore, it is important to analyze how this randomness affects the algorithm.

For this purpose, a preliminary parameter for the tabu length was selected with the class recommendation of a value between 7 and 20. With this in mind, a value of 10 was chosen.

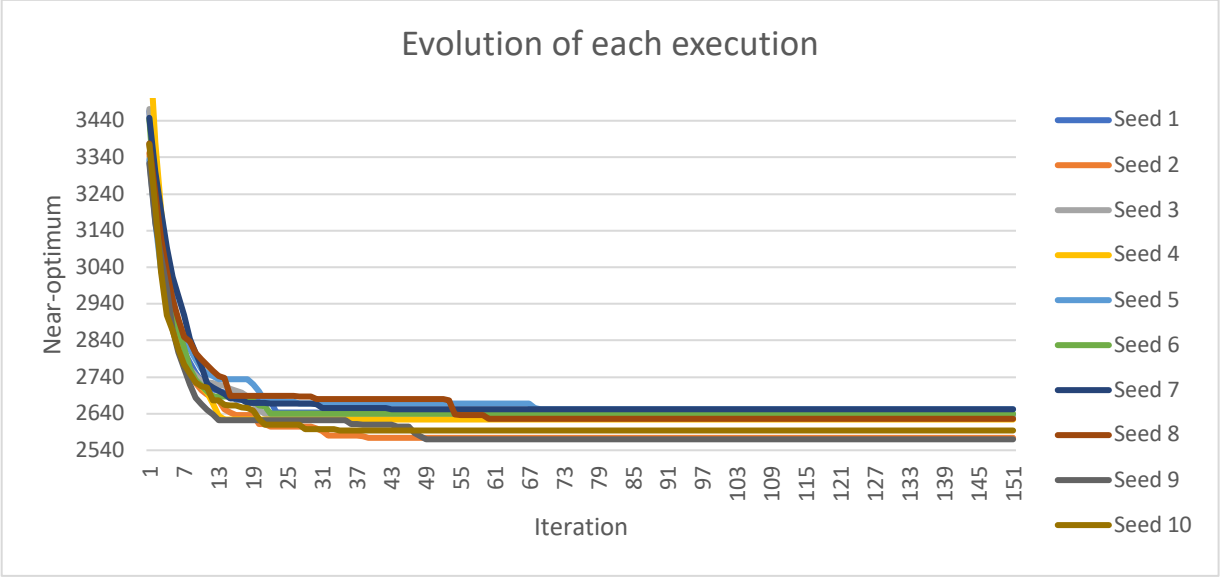
Results

The following graphs show the near-optimums found in the execution of the canonical tabu search with a tabu list size of 10 for 10 different seeds (that define only the initial solution).



Graph 1: Reported near-optimums for the preliminary runs

Graph 1 shows that the initial solution is important for the overall performance of the algorithm. For seed 2 and seed 9 better results were obtained than for all the other seeds. In orange the graph also shows the mean of the results obtained and in gray the global optimum for visualization purposes. For these results, the global optimum was found only with seed 9 at iteration 48. In the following graph, the evolution of the reported near-optimum by seed is described (for visualization purposes, the graph only shows until iteration 150).



Graph 2: Evolution of near-optimum by seed

Graph 2 shows that the convergence speed was similar for all the seeds tested, only differing in the near-optimum reported at the end of the iterations.

Conclusions of the preliminary runs

From the previous results, it can be concluded that the initial solution of the algorithm influence in which sections of the search space the algorithm explores and the direction in which the algorithm is led. For these results, only two seeds reached near-optimal results (or optimal in one case). Considering that the only difference in the execution of these outliers was the initial solution, then it can be concluded that the algorithm is very sensitive to it and that simply restarting the execution from another starting point can be a good strategy for improving the results obtained.

In terms of the convergence speed, the different seeds reported different near-optimal solutions but no different convergence speeds. Graph 2 shows that for all seeds, the algorithm reached a good level of convergence between iteration 50 and 70. From this, it can be concluded that the deterministic operation of the algorithm does not suffer many changes from the quality of the starting solution.

Tabu length tuning runs

Definitions

The main parameter to be tuned in tabu search (other than important definitions as the aspiration criterion) is the length of the tabu list. This parameter has a great influence on how much the algorithm will restrict its search. If a greater length is selected, then the restricted moves will last longer in the operation of the tabu search and will have more chances to defy improving moves. If the tabu length is shorter, then the algorithm will restrict fewer moves and will act more like a greedy search.

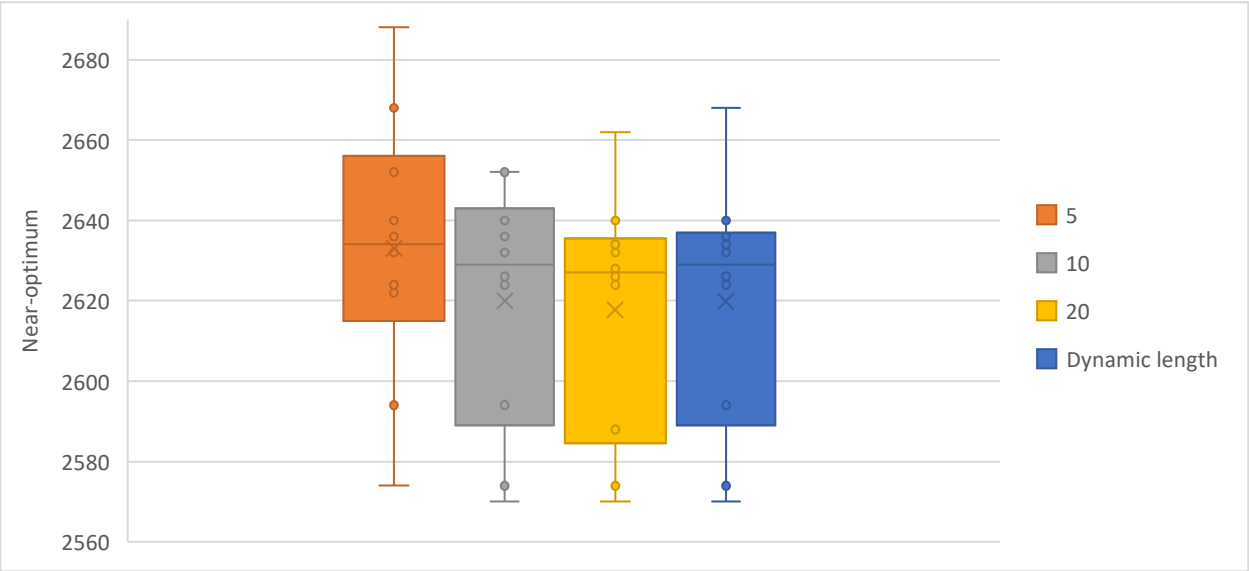
To explore the influence of this parameter in the results of the algorithm, different levels for this parameter were executed for the same 10 seeds. These levels were defined considering the class recommendation and changing the initial chosen value of 10 down and up. Also, a random and dynamic length for the tabu search was implemented. This dynamic implementation activates every 20 iterations of the algorithm, defining a new tabu length for the next iterations. The different levels for this parameter and the random distribution of the dynamic tabu length are described in the next table.

Table 2: Levels for the tabu length testing

Implementation of tabu length	Values	Justification
Fixed length	[5,10,20]	Range considering class recommendation interval
Dynamic length	Uniform (5,20)	Random uniform with the same range as the levels.

Results

For each of the four options to define the length of the tabu list ten seeds were executed. The results of these runs are shown in the boxplots of the following graph.



Graph 3: Boxplots of the different options for tabu list length

The fact that all the boxplots' intervals clearly overlap with each other led to the conclusion that more testing (bigger sample) is required or that the different options tested are equivalent. Also, in terms of convergence speed, the different levels had an almost equivalent behavior (as shown in appendix 2). In the next table the average results for the executions of 10 seeds are shown for each of the levels experimented.

Table 3: Average results of tabu list length experimentation

Level	Average near-optimum reported
Dynamic length	2619.8
5	2633
10	2620
20	2617.8

Conclusions of the tabu length tuning runs

The previous results showed that tabu length for the tested levels was not significant. Having said this, there was a trend in which a higher tabu length resulted in better results. This may indicate that further testing with larger values would improve the algorithm performance and give significant differences.

Another observation that can be made is the fact that, except for the 5 length, the variance of the results was similar between each level. This can be explained by the fact that the initial random solution has a great impact on the performance of the algorithm and considering that the tabu length was not significant, this aspect of the algorithm was the only source of variation in the obtained results.

Considering that for the executed runs the difference in terms of reported near-optimums and convergence speeds was not significant, the value of 20 for the tabu list length was chosen for the next executions of the algorithm. This, taking into account that it reached the best average of all the tested levels.

Aspiration criteria, partial neighborhood and diversification runs

Definitions

For this section, three different variations of the algorithm will be tested: the implementation of an aspiration criterion, the exploration of less than the full neighborhood for each iteration and the incorporation of diversification mechanisms in the search. For this, some definitions must be made to describe the implemented variations.

The first one is the criteria for the aspiration criteria used. In this case the selected way to implement the aspiration criteria was *best solution so far*. This way the algorithm will ignore the tabu list only when the possible move results in a better improvement in the objective function than all the previous evaluated values.

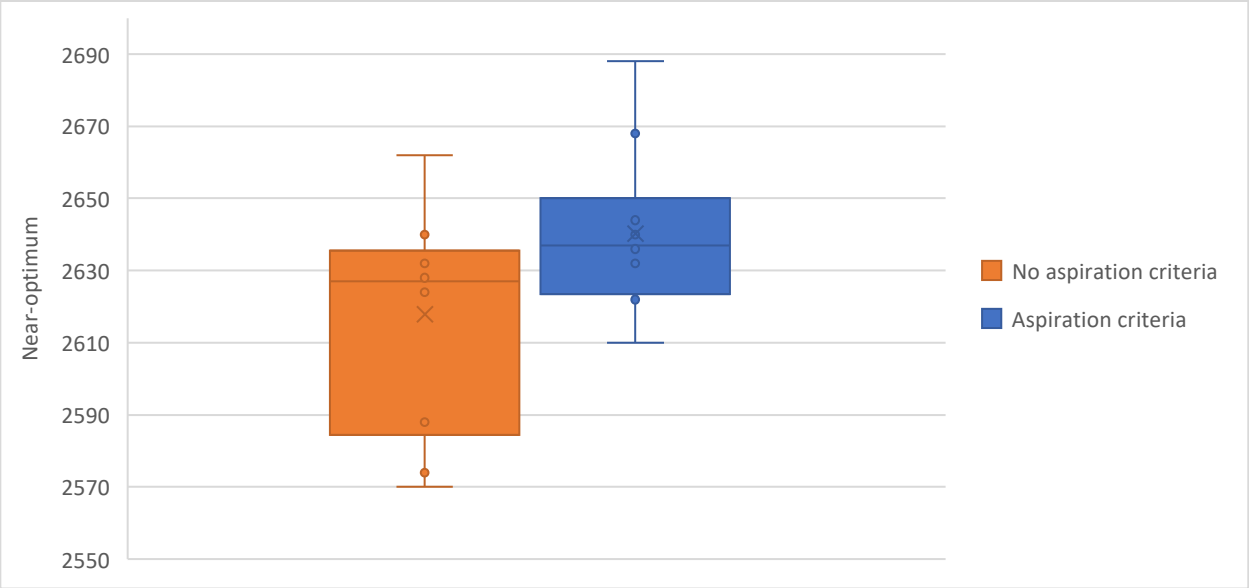
For the partial neighborhood evaluation, only half of the possible neighbors were evaluated. This was implemented by selecting only half of the possible indexes for the possible permutation at random. The number of chosen indexes was fixed on the value of 10 (from the 20 nodes) and was selected with a discrete uniform distribution. This way, a random operator to select which permutation to explore was implemented.

Finally, for the diversification strategies, two implementations were tested. These were backtracking to elite solution and restarting at a random solution, both options maintaining the tabu list from previous iterations. The executed backtracking operates by keeping track of all solutions that had been saved as the best solution so far at some point of the search and then selecting one at random (uniformly). With this solution the algorithm continues the next iteration. The second method considers a whole new random solution for the next iteration of the algorithm. Both strategies activate when no improvement is achieved in 30 consecutive iterations.

Results

Aspiration criteria

For the aspiration criteria, the same 10 seeds were implemented and tested. The boxplots of the results are shown in the following graph. Also, for comparison purposes, the same configuration of the algorithm but without aspiration criteria is shown.



Graph 4: Boxplots with and without aspiration criteria

Graph 4 shows that the aspiration criteria reported a worse performance overall than the equivalent without this feature. With no aspiration criteria the global optimum was reached one time for a given seed, while in the runs with aspiration criteria the global optimum was never reached.

To explain this the following table is presented.

Table 4: Consolidated results of the aspiration criteria runs

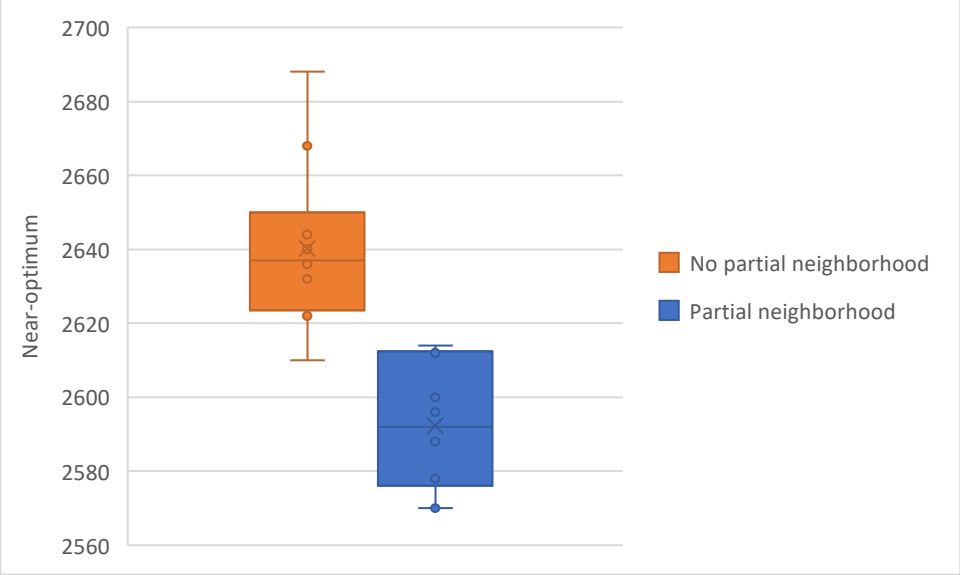
	Mean	Standard deviation
Aspiration	2640.2	22.7732787
No Aspiration	2617.8	30.2059597

Table 4 shows that in fact the implementation with no aspiration criteria performed better in terms of average reported near-optimums. To explain this behavior, the standard deviation was also calculated for each of the results. For the results with aspiration criteria, the standard deviation was much lower than the equivalent with no aspiration criteria. This consistency partially explains why this variation of the algorithm underperformed. Having a possibility to ignore the tabu search causes that the algorithm, when exploring a certain solution, move to it in every single execution of the algorithm.

Even though this is positive, it caused that the runs for the different seeds meet a certain common point, homologating the search for all seeds and reducing the effects of the starting solution. Given the executed runs, it seems that this common point in this case made the algorithm reach a local optimum. The full results of these runs can be found in appendix 3.

Partial neighborhood

The previously explained partial neighborhood strategy was executed for the same 10 seeds implemented over this homework. This strategy was added over the implementation of the aspiration criteria with the selected tabu length value (20). Considering this, the runs that implemented the aspiration criteria were selected for a proper comparison. The following graphs show the boxplots for the results of implementing a partial neighborhood.



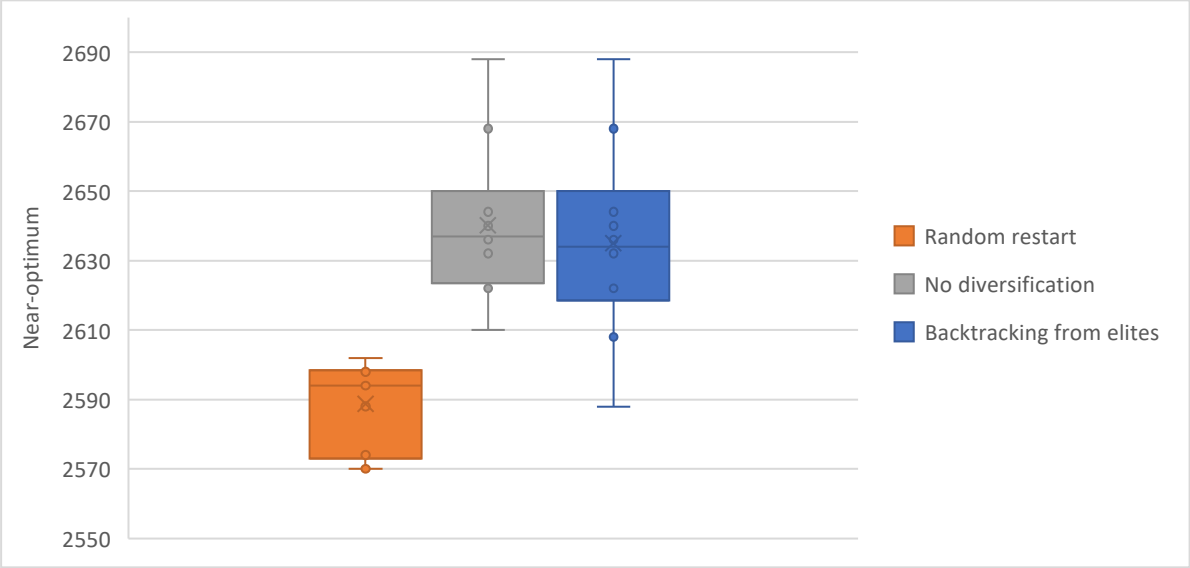
Graph 5: Boxplots with and without partial neighborhood

Graph 5 shows that adding a partial neighborhood significantly improves the performance of the algorithm. This conclusion can be made considering that the boxplots' intervals do not overlap with each other. This can be explained by two aspects. The first one is the omission of certain solution that causes the algorithm to get stuck in local optimums. The second explanation is the diversification that the random operator added to the algorithm.

The results of each seed of the partial neighborhood runs can be found in appendix 3.

Diversification strategies

As mentioned before, two diversification strategies were implemented: the backtracking to elite solutions (chosen at random) and the restart from a completely random solution. As required by the homework, these strategies were added over the implementation of the aspiration criteria. Considering this, the proper comparison point would be the runs with aspiration criteria. Graph 6 shows the boxplots of the results of the diversification and the defined comparison point.



Graph 6: Boxplots with and without diversifications strategies

The previous graph shows that the backtracking to elites and the runs only with aspirations criteria were similar in near-optimums reported. On the other hand, the random restart strategy had great improvements compared to the other two options, even reaching the global optimum.

It can be noted that the backtracking to elite solutions and the aspiration criteria runs are related in the sense of overrepresenting the best solutions found in the search. This can make that the algorithm get stuck in the same local optimums and don't reach better solutions.

For the random restart (maintaining the tabu list), the results were clearly better. Even though the aspiration criteria reduce the effects of the initial solution, it seems that having multiple restarts lets the algorithm explore in a wider way the search space, avoiding local optimums.

Conclusions of the Aspiration criteria, partial neighborhood and diversification runs

As a conclusion for this section, several observations can be made. The first one is how the aspiration criteria acted as ignoring the restriction given by the tabu list. In this implementation on this problem, the aspiration criteria had a negative effect in terms of near-optimums reported and exploration of the search space. The aspiration criteria act by ignoring the operator that avoids intensification in the search (tabu list) and so it is reasonable to say that this feature may harm the search for optimums by rising the greedy behavior of the algorithm. Having said this, even though in general it is a positive idea to implement this kind of feature, it must be added with effective diversification strategies to overcome the excessive intensification5800.

It is also interesting to note how the aspiration criteria reduced the excessive effects of the random seed in the performance of the algorithm. As seen in the preliminary runs, the algorithm is very sensitive to the random solution but also only two seeds performed over the mean of the results (graph 1). This behavior justifies the implementation of the aspiration criteria even though

the mean performance is reduced. If accompanied by diversification strategies, the aspiration criteria may reach better and more consistent results than the other implementations.

For the partial neighborhood strategy, the results were improved in a significant way. The fact that it was incorporated with a random operator added variability to the algorithm and let it explore sections of the search that the deterministic implementation probably would never reach.

Finally, for the diversification strategies, it is interesting that the random selection of an elite solution and backtracking didn't improve the algorithm's performance. This can be explained by the fact that the tabu list, even though it was kept over the following restart, it wasn't big enough to make a difference from previous paths. It is possible that the algorithm just went over the same route it took in the first place after the restart.

On the other hand, the complete random restart had good effects on the reported solutions. Even though the tabu list size was the same for the elite-restart, the fact that it explored a whole new section of the search space made it find new paths with better solutions.

General conclusions

Over this report tabu search was executed for 10 seeds for each implementation (80 runs in total) and had proven to be a good meta-heuristic for combinatorial problems. For most of the implementations on this report, the global optimum of 1570 was reached at least for one executed seed. While the algorithm is very sensitive to the initial solution of the iterations, an implementation that considers this aspect can reach the global optimum in few very few iterations.

Overall, this report and the corresponding experimentation, need of tabu search to diversify the search was clear in the results. While the algorithm is good in local search (intensification), the sensitivity to the initial solution and the structure of the algorithm creates a lack of diversification that affects the overall performance of the algorithm. This can be easily taken into account by implementing diversification strategies that fix this issue of the algorithm. In this specific case, the strategies that had the best effects in this sense were the random partial neighborhood and the restart from a random solution. These two strategies improved greatly the performance of the algorithm over an implementation that raised intensification such as the aspiration criteria.

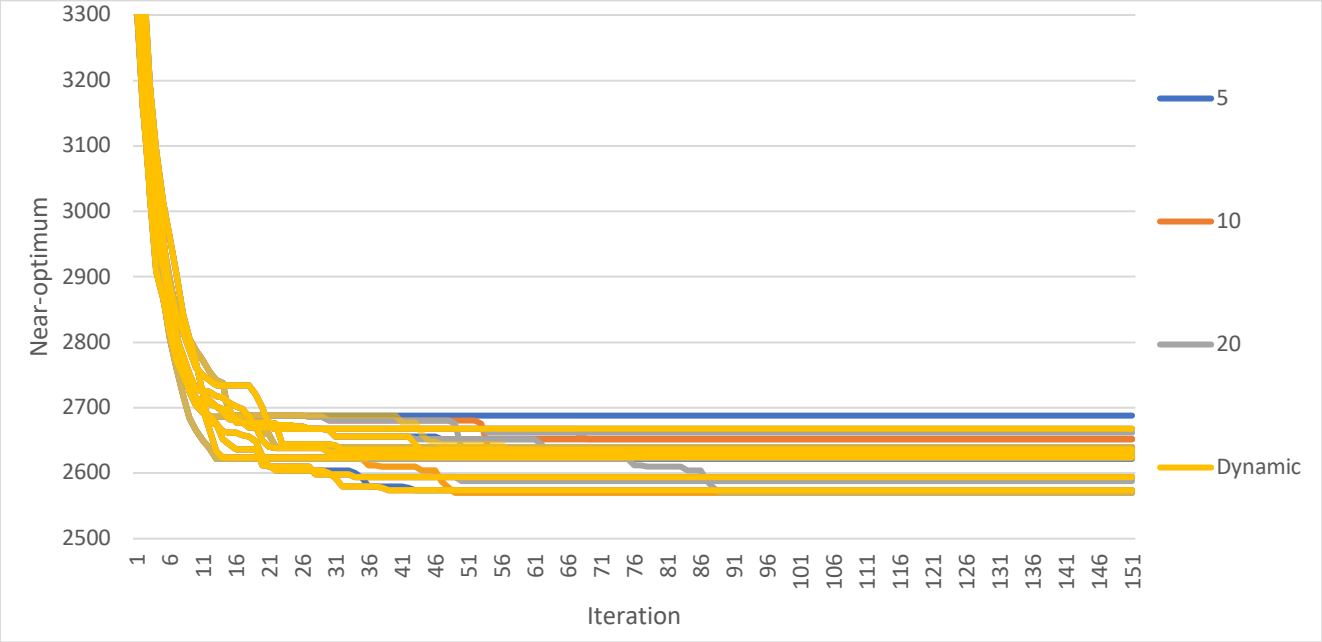
Finally, in terms of convergence speed, the implementations tested didn't show bigger differences. This is a sign of a consistent behavior and the mainly deterministic operators that the algorithm implements. It is interesting to note this considering that other meta-heuristics in the literature rely heavily on randomness for exploring the search space. Therefore, this aspect of tabu search may be a specific feature that can be exploited in specific implementation and problems.

Appendix 1: Results of the preliminary runs

Table 5: Preliminary run results

Seed	Tabu list length	Iterations	Neighborhood	Diversification	Aspiration criterion	Solutions reported	Near-optimum
571024	10	500	complete	none	none	[17. 13. 19. 5. 3. 1. 11. 2. 16. 14. 10. 12. 0. 18. 7. 15. 4. 9. 8. 6.]	2636
1130205	10	500	complete	none	none	[3. 16. 18. 5. 2. 4. 1. 7. 19. 13. 8. 12. 9. 17. 11. 14. 0. 15. 10. 6.]	2574
1048596	10	500	complete	none	none	[11. 8. 10. 3. 17. 18. 13. 7. 15. 16. 2. 12. 19. 6. 9. 1. 0. 5. 4. 14.]	2632
3372329	10	500	complete	none	none	[1. 13. 5. 14. 16. 10. 11. 12. 0. 6. 2. 7. 15. 8. 18. 3. 4. 9. 19. 17.]	2624
4530806	10	500	complete	none	none	[2. 17. 0. 16. 8. 4. 7. 13. 5. 6. 11. 12. 9. 10. 18. 15. 3. 1. 19. 14.]	2652
5343724	10	500	complete	none	none	[16. 8. 18. 4. 11. 15. 6. 12. 5. 17. 1. 2. 10. 3. 13. 0. 19. 14. 9. 7.]	2640
12098115	10	500	complete	none	none	[15. 12. 14. 5. 3. 9. 10. 1. 19. 18. 16. 11. 4. 13. 7. 17. 0. 8. 2. 6.]	2652
7348126	10	500	complete	none	none	[18. 6. 19. 0. 13. 9. 8. 7. 14. 17. 5. 12. 4. 11. 2. 10. 15. 16. 1. 3.]	2626
32642329	10	500	complete	none	none	[18. 6. 3. 5. 16. 19. 17. 13. 4. 2. 8. 7. 14. 1. 11. 9. 15. 0. 10. 12.]	**2570**
412106	10	500	complete	none	none	[3. 6. 0. 18. 9. 14. 8. 12. 10. 2. 17. 7. 19. 5. 11. 15. 4. 1. 16. 13.]	2594

Appendix 2: Results of the parameter tuning runs



Graph 7: Evolution of the near-optimum reported by iteration

Appendix 3: Results of the variations runs

Table 6: Results of the aspiration criteria runs for 10 seeds

Seed	Tabu list length	Iterations	Neighborhood	Diversification	Aspiration criterion	Solutions reported	Near-optimum
571024	20	500	complete	none	best	[2. 13. 19. 5. 3. 1. 7. 11. 16. 9. 10. 17. 0. 18. 12. 15. 4. 14. 8. 6.]	2644
1130205	20	500	complete	none	best	[8. 16. 9. 5. 2. 3. 7. 1. 19. 13. 12. 17. 4. 18. 11. 14. 0. 15. 10. 6.]	2636
1048596	20	500	complete	none	best	[11. 8. 10. 3. 17. 18. 13. 7. 15. 16. 2. 12. 19. 6. 9. 1. 0. 5. 4. 14.]	2632
3372329	20	500	complete	none	best	[1. 13. 5. 14. 16. 10. 11. 12. 0. 6. 2. 7. 15. 8. 18. 3. 4. 9. 19. 17.]	2624
4530806	20	500	complete	none	best	[7. 17. 1. 10. 9. 3. 8. 13. 0. 2. 11. 12. 4. 16. 18. 15. 5. 6. 19. 14.]	2668
5343724	20	500	complete	none	best	[16. 8. 18. 4. 11. 15. 6. 12. 5. 17. 1. 2. 10. 3. 13. 0. 19. 14. 9. 7.]	2640
12098115	20	500	complete	none	best	[1. 13. 4. 10. 5. 9. 11. 6. 19. 3. 16. 17. 14. 18. 7. 15. 0. 2. 8. 12.]	2638
7348126	20	500	complete	none	best	[18. 1. 17. 5. 13. 14. 3. 7. 19. 12. 6. 8. 9. 11. 2. 16. 15. 10. 0. 4.]	2688
32642329	20	500	complete	none	best	[18. 6. 2. 10. 16. 17. 13. 12. 4. 1. 14. 8. 19. 3. 7. 9. 15. 0. 5. 11.]	2622
412106	20	500	complete	none	best	[18. 1. 10. 3. 9. 14. 8. 12. 15. 11. 17. 6. 19. 5. 7. 16. 4. 0. 2. 13.]	2610

Table 7: Results of the partial neighborhood runs for 10 seeds

Seed	Tabu list length	Iterations	Neighborhood	Diversification	Aspiration criterion	Solutions reported	Near-optimum
571024	20	500	random	none	best	[3. 16. 18. 5. 8. 4. 2. 6. 19. 13. 7. 12. 9. 17. 11. 14. 0. 15. 10. 1.]	2580
1130205	20	500	random	none	best	[5. 12. 19. 16. 0. 4. 1. 6. 9. 14. 11. 8. 3. 13. 7. 10. 15. 18. 17. 2.]	2614
1048596	20	500	random	none	best	[6. 13. 16. 4. 10. 1. 3. 7. 15. 11. 9. 8. 0. 18. 12. 19. 5. 17. 14. 2.]	2612
3372329	20	500	random	none	best	[1. 14. 4. 17. 16. 5. 6. 12. 0. 3. 7. 8. 10. 9. 13. 2. 15. 19. 18. 11.]	2614
4530806	20	500	random	none	best	[18. 11. 15. 1. 14. 19. 13. 3. 5. 17. 2. 12. 9. 10. 7. 0. 4. 16. 6. 8.]	2588
5343724	20	500	random	none	best	[1. 13. 16. 14. 3. 0. 2. 6. 15. 17. 11. 12. 5. 18. 8. 10. 4. 19. 9. 7.]	**2570**
12098115	20	500	random	none	best	[18. 6. 10. 3. 9. 14. 13. 7. 15. 5. 17. 12. 19. 11. 2. 16. 4. 0. 1. 8.]	2596
7348126	20	500	random	none	best	[18. 7. 9. 5. 13. 19. 17. 11. 4. 8. 10. 12. 14. 2. 6. 0. 15. 3. 1. 16.]	2600
32642329	20	500	random	none	best	[16. 8. 1. 9. 18. 15. 17. 11. 0. 2. 6. 7. 10. 3. 13. 5. 19. 4. 14. 12.]	**2570**
412106	20	500	complete	none	best	[18. 1. 10. 3. 9. 14. 8. 12. 15. 11. 17. 6. 19. 5. 7. 16. 4. 0. 2. 13.]	2610